CS4782 Deep Learning Project Proposal

Team

Kevin Tang - qt58
Madur Malliah - mr2466
Weitao Su - ws535

1. Paper Selection

1.1 Title, authors, and publication venue of the chosen paper

Title: Low-Rank Adaptation of Large Language Models (LoRA)
Authors: Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen
Venue: arXiv preprint, 2021 (arXiv:2106.09685)

1.2 Brief summary of the chosen paper

At the time when LoRA was released, one of the biggest bottlenecks in using pretrained model was the compute and memory cost of fine-tuning the billions of parameters in models such as GPT-2. To combat this, there had been a couple of other parameter-efficient fine-tuning methods prior to LoRA, such as adapter layers and prefix tunings. However, these had their drawbacks such as adapter layers incurring additional inference latency, and prefix tuning not giving us direct control over model weights, resulting in lesser control over the model adaptation.

LoRA is a novel parameter-efficient fine-tuning method for fine-tuning pretrained neural networks. Instead of updating all pretrained model weights, LoRA freezes the base model and learns low-rank update matrices inserted into selected linear layers. This greatly reduces trainable parameter count and memory usage while preserving strong downstream performance. The experiments done by the author shows that LoRA is able to achieve similar or even better performance than traditional full fine-tuning methods, while only relying on much fewer training parameters.

1.3 Brief justification of why you chose this paper

We chose this paper because it is highly relevant to modern deep learning workflows, and balances strong theory intuition with practical impact. It gives us insight into one of the most common parameter-efficient fine-tuning methods used, and let us better understand how parameters such as rank and module affect the efficiency and accuracy of the model.

Additionally, it is also feasible to reproduce this project at a class-project scale, as the datasets used and model sizes are manageable with our available resources. This means that we can also do meaningful experimentation on quality-vs-efficiency tradeoffs.

2. Data

2.1 Prove that the data exists/is accessible

We will use publicly available NLP benchmark datasets (GLUE tasks such as SST-2, MRPC, and RTE), accessible through Hugging Face Datasets and official benchmark sources. Pretrained model checkpoints (e.g., GPT-2, RoBERTa-base) are publicly accessible via Hugging Face Transformers. Thus both data and model resources are fully available and reproducible.

3. Results Selection

3.1 Tell us which results you want to replicate

Replicate core claim on selected tasks:

1. Compare full fine-tuning vs LoRA on SST-2, MRPC, RTE
2. Rank sweep test on performance (r in {2,4,8,16,32})
3. Report:
    - task metric (accuracy/F1)
    - trainable parameter count
    - peak GPU memory
    - throughput (samples/sec)

Goal: Verify that LoRA reaches near full fine-tuning quality at much lower training cost.

Possible extension work:

1. Target module sweep (q,v) vs (q,k,v,o)
2. Seed robustness on at least one task (3 seeds)
3. Learning dynamics diagnostics:

- training/validation curves
- memory and speed tradeoff curves
- performance vs trainable-params frontier

Goal: understand when and why LoRA works (not only whether it works).

4. Re-implementation Plan

4.1 Describe Architecture

Backbone: GPT-2 M
LoRA adapters added to attention projection layers
Base model frozen during LoRA runs
Full fine-tuning baseline updates all parameters

4.2 Describe Method

The general pipeline for re-implementation of LoRA is as follows:

1. Load dataset/task with standard split and tokenizer
2. Implement two training modes:
    - full fine-tuning baseline
    - LoRA fine-tuning
3. Keep training protocol matched where possible (epochs, batch size, optimizer family, max length)
4. Log performance + compute metrics
5. Rank ablation runs

Methodology Plan for possible extension work**:**

- Module ablation runs
- Multi-seed robustness run for one representative task

4.3 Describe Metrics

- Task quality: accuracy/F1 (task-dependent: SST-2, MRPC, RTE)
- Efficiency:
- trainable parameter count
- peak GPU memory
- throughput (samples/sec or tokens/sec)

For possible extension work:

- variance across seeds
- Pareto tradeoff curves (quality vs efficiency)

4.4 Details about how much compute and time are required to replicate results

Core re-implementation:

- Team effort: ~25–40 total hours
- GPU: ~20–40 GPU-hours
- Calendar: ~5–8 days

If executing the possible extension work:

- Team effort: ~45–80 total hours
- GPU: ~35–70 GPU-hours
- Calendar: ~10–14 days

—- NOTES SECTION —-

3. Results Selection

3.1 Tell us which results you want to replicate

Plan A (Grade-safe / lower risk)

Replicate core claim on selected tasks:

4. Compare full fine-tuning vs LoRA on SST-2, MRPC, RTE
5. Report:
    - task metric (accuracy/F1)
    - trainable parameter count
    - peak GPU memory
    - throughput (samples/sec)

Goal: verify that LoRA reaches near full fine-tuning quality at much lower training cost.

Plan B (Max-learning / deeper)

Do Plan A plus hypothesis-driven ablations:

4. Rank sweep (r \in {2,4,8,16,32})
5. Target module sweep (q,v) vs (q,k,v,o)
6. Seed robustness on at least one task (3 seeds)
7. Learning dynamics diagnostics:

- training/validation curves
- memory and speed tradeoff curves
- performance vs trainable-params frontier

Goal: understand when and why LoRA works (not only whether it works).

4. Re-implementation Plan

4.1 Describe Architecture

Backbone: RoBERTa-base (or similar base-size transformer)
LoRA adapters added to attention projection layers
Base model frozen during LoRA runs
Full fine-tuning baseline updates all parameters

4.2 Describe Method

Common pipeline for both plans:

3. Load dataset/task with standard split and tokenizer
4. Implement two training modes:
    - full fine-tuning baseline
    - LoRA fine-tuning
6. Keep training protocol matched where possible (epochs, batch size, optimizer family, max length)
7. Log performance + compute metrics

Additional for Plan B:

- Rank/module ablation runs
- Multi-seed robustness run for one representative task

4.3 Describe Metrics

- Task quality: accuracy/F1 (task-dependent)
- Efficiency:
- trainable parameter count
- peak GPU memory
- throughput (samples/sec or tokens/sec)
- For Plan B:
- variance across seeds
- Pareto tradeoff curves (quality vs efficiency)

4.4 Details about how much compute and time are required to replicate results

Plan A (Grade-safe)

- Team effort: ~25–40 total hours
- GPU: ~20–40 GPU-hours
- Calendar: ~5–8 days
- Risk level: low-moderate

Plan B (Max-learning)

- Team effort: ~45–80 total hours
- GPU: ~35–70 GPU-hours
- Calendar: ~10–14 days
- Risk level: moderate-high (more experiment/debug overhead)

Both are feasible; Plan B is better for learning depth.